Project Report

1.Title : SecureShare -A cloud-based Secure File Sharing System

Deployed on LEMP Cloud Infrastructure

2.Student Details

Name : Sanika Rahul Nimbalkar

Roll no: B31058

Course : AWS with Python

College : Zeal college of Engineering and Research

Table of Content

1.Title..... 1

2.Student details……..1

3. Introduction 3

4. Problem Statement 3

5. Objectives 3

6. Cloud Service Model 4

7. Deployment Model 6

8. Tools & Technologies 6

9. System Architecture 6

10. Implementation 7

11. Results & Output 9

12. Advantages 9

13. Future Scope 9

14. Conclusion 10

15. References 10

3.Introduction

Cloud computing is a modern computing paradigm that enables users to access computing resources such as servers, storage, databases, and applications over the internet without the need for direct physical infrastructure. It eliminates the need for maintaining local servers and provides scalability, flexibility, and cost efficiency. Organizations and individuals can deploy applications globally with minimal setup using cloud platforms.

In today's digital era, secure data sharing is a critical requirement. Traditional file sharing methods such as email attachments or physical storage devices lack proper security mechanisms and often fail to provide controlled access. This leads to issues like unauthorized access, data leakage, and lack of traceability. Cloud computing solves these problems by enabling centralized, secure, and controlled access to data.

This project, SecureShare, is a cloud-based file sharing system designed and deployed using a LEMP stack (Linux, Nginx, MySQL/MariaDB, PHP) on an AWS EC2 instance. It allows users to upload files with password protection and expiry time, ensuring that files are accessible only for a limited duration. Additionally, the system includes a request-based access mechanism, where users can request access after expiry and the owner can approve it.

The system demonstrates how cloud infrastructure can be used to build scalable and secure applications. It also showcases real-world deployment using Infrastructure as a Service (IaaS), making it highly relevant in modern cloud computing environments.

4.Problem Statement

In traditional file sharing systems, there is a lack of security, control, and flexibility. Users often share files through email or messaging platforms without any control over who can access the file and for how long. Once shared, files remain accessible indefinitely, increasing the risk of unauthorized access and data misuse.

Another major problem is the absence of controlled access mechanisms. There is no way to enforce password protection or expiry-based access in many basic systems. Additionally, once access is lost or restricted, there is no structured way to request access again.

This project addresses these issues by providing a secure, cloud-based file sharing platform that incorporates password protection, expiry time, and request-based access control. It ensures that files are shared securely and access is granted only under controlled conditions.

5.Objective:

The primary objective of this project is to design and implement a secure and efficient cloud-based file sharing system using modern web technologies and cloud infrastructure. The system aims to overcome the limitations of traditional file sharing methods by incorporating security, control, and accessibility features.
One of the main goals of the project is to provide secure file sharing by implementing password protection. Each uploaded file is associated with a passcode, ensuring that only authorized users can access it. This enhances data privacy and prevents unauthorized downloads.

Another important objective is to implement an expiry-based access control mechanism. Files are not available indefinitely; instead, the user can define a time duration after which the file becomes inaccessible. This ensures temporary sharing and reduces the risk of long-term exposure of sensitive data.

The project also aims to introduce a request-based access system. When a file expires, users can request access again, and the file owner or admin can approve the request. Upon approval, access is granted for a limited duration.

This adds an additional layer of controlled access and flexibility. A key objective of this project is to demonstrate real-world cloud deployment using the LEMP stack on AWS EC2. This includes setting up a Linux server, configuring Nginx, installing PHP, and managing a database using MariaDB.

Additionally, the project focuses on user experience (UI/UX) by designing a modern, responsive interface using Bootstrap and custom CSS. Features such as file selection preview, animations, and clear feedback messages improve usability .Finally, the project aims to provide hands-on experience in cloud computing concepts, including Infrastructure as a Service (IaaS), server configuration, and application deployment.

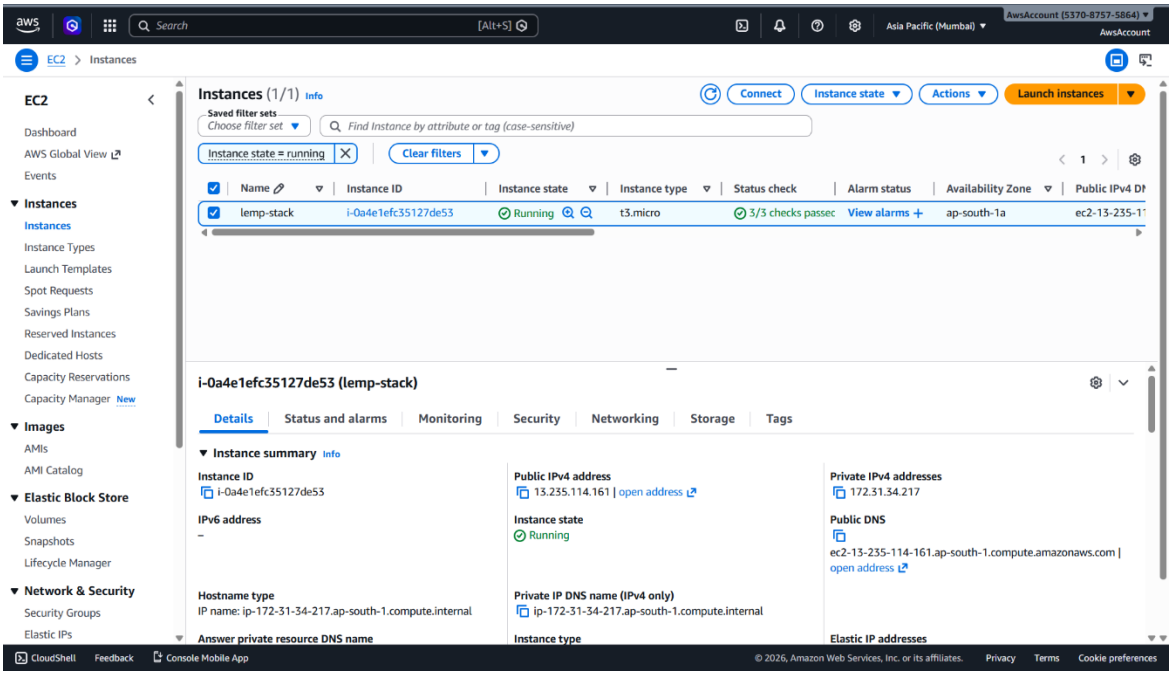
6. Cloud Service Model

In this project, an AWS EC2 instance is used as the core infrastructure. The EC2 instance acts as a virtual server running a Linux operating system. Unlike Platform as a Service (PaaS), where much of the environment is managed automatically, IaaS provides full control over the server configuration, allowing the user to install and manage required software components.

The LEMP stack is installed manually on the EC2 instance, which includes:

- Linux as the operating system
- Nginx as the web server
- PHP for backend processing
- MariaDB as the database

This approach provides flexibility in configuring the server according to application requirements. The user is responsible for installing, configuring, and maintaining the software stack .The use of IaaS allows the project to simulate real-world deployment scenarios where developers manage infrastructure directly. It also enables scalability, as the EC2 instance can be resized or replicated based on demand.
Thus, the project effectively demonstrates how cloud infrastructure can be utilized to deploy and manage web applications.



1.1 creating a ec2 instance for LEMP stack server deployment

```
ec2-user@ip-172-31-34-217:/usr/share/nginx/html

Dell@DESKTOP-AAB5LU3 MINGW64 ~
$ cd downloads/

Dell@DESKTOP-AAB5LU3 MINGW64 ~/downloads
$ ssh -i lempStack.pem ec2-user@13.235.114.161

A newer release of "Amazon Linux" is available.
  Version 2023.10.20260330:
Run "/usr/bin/dnf check-release-update" for full release and version update info
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Mon Apr  6 04:59:21 2026 from 163.223.250.37
[ec2-user@ip-172-31-34-217 ~]$
[ec2-user@ip-172-31-34-217 ~]$ nginx -v
nginx version: nginx/1.28.2
[ec2-user@ip-172-31-34-217 ~]$ php -v
PHP 8.5.3 (cli) (built: Feb 10 2026 18:25:51) (NTS gcc x86_64)
Copyright (c) The PHP Group
Built by Amazon Linux
Zend Engine v4.5.3, Copyright (c) Zend Technologies
    with Zend OPcache v8.5.3, Copyright (c), by Zend Technologies
[ec2-user@ip-172-31-34-217 ~]$ mysql --version
mysql  Ver 15.1 Distrib 10.5.29-MariaDB, for Linux (x86_64) using  EditLine wrapper
[ec2-user@ip-172-31-34-217 ~]$ sudo systemctl restart nginx
[ec2-user@ip-172-31-34-217 ~]$ sudo systemctl restart php-fpm
[ec2-user@ip-172-31-34-217 ~]$ cd /usr/share/nginx/html
[ec2-user@ip-172-31-34-217 html]$ ls
404.html  assets        fileshare.zip    icons      initialFile.php  nginx-logo.png  requests.php
50x.html  download.php  finalupload.php  index.php  navbar.php       poweredby.png   uploads
[ec2-user@ip-172-31-34-217 html]$ |
```

1.2 SSH connection to server and installing the services (LEMP stack)

```
ec2-user@ip-172-31-34-217:/usr/share/nginx/html

MariaDB [(none)]> show tables;
ERROR 1046 (3D000): No database selected
MariaDB [(none)]> use fileshare;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [fileshare]> show tables;
+---------------------+
| Tables_in_fileshare |
+---------------------+
| files               |
| requests            |
+---------------------+
2 rows in set (0.000 sec)

MariaDB [fileshare]> describe files;
+-------------+--------------+------+-----+---------+----------------+
| Field       | Type         | Null | Key | Default | Extra          |
+-------------+--------------+------+-----+---------+----------------+
| id          | int(11)      | NO   | PRI | NULL    | auto_increment |
| filename    | varchar(255) | YES  |     | NULL    |                |
| filepath    | text         | YES  |     | NULL    |                |
| expiry_time | datetime     | YES  |     | NULL    |                |
| password    | varchar(255) | YES  |     | NULL    |                |
| passcode    | varchar(255) | YES  |     | NULL    |                |
+-------------+--------------+------+-----+---------+----------------+
6 rows in set (0.001 sec)

MariaDB [fileshare]> describe requests;
+---------+-------------+------+-----+---------+----------------+
| Field   | Type        | Null | Key | Default | Extra          |
+---------+-------------+------+-----+---------+----------------+
| id      | int(11)     | NO   | PRI | NULL    | auto_increment |
| file_id | int(11)     | YES  |     | NULL    |                |
| status  | varchar(20) | YES  |     | pending |                |
+---------+-------------+------+-----+---------+----------------+
3 rows in set (0.001 sec)

MariaDB [fileshare]> |
```

1.3 creating Database on MYSQL

7.Deployment Model

The deployment model used in this project is the Public Cloud model. In a public cloud environment, computing resources are owned and operated by a third-party cloud service provider, such as AWS, and are made available to users over the internet . In this project, the application is hosted on an AWS EC2 instance, which is accessible through a public IP address. This means that the application can be accessed from any device with an internet connection, making it highly scalable and flexible .The public cloud model provides several advantages, including cost-effectiveness, ease of deployment, and global accessibility. There is no need to invest in physical hardware, as all resources are provisioned virtually.

The application is deployed in such a way that the web server (Nginx) listens on port 80, allowing HTTP requests from users. Security groups in AWS are configured to allow inbound traffic for HTTP and SSH, ensuring secure and controlled access. The use of a public cloud model also allows for future enhancements such as domain integration, load balancing, and scaling.

8.Tools and Technologies

The project utilizes a combination of cloud services, backend technologies, and frontend tools to build a complete and functional system.

Cloud Platform: AWS EC2 is used to host the application. It provides a scalable virtual server environment where the LEMP stack is installed and configured.

Operating System: Linux is used as the base operating system. It provides stability, security, and compatibility with server applications.

Web Server: Nginx is used as the web server. It handles incoming HTTP requests and serves static and dynamic content efficiently.

Backend Language: PHP is used for server-side scripting. It handles file uploads, database interactions, authentication, and business logic.

Database: MariaDB (MySQL-compatible) is used to store file metadata such as filename, file path, passcode, expiry time, and request status.

Frontend Technologies: HTML, CSS, and Bootstrap are used to design a responsive and user-friendly interface. Custom CSS is added to enhance visual appearance .

Development Tools: Code editors, terminal (SSH), and browser tools are used during development and deployment.

9.System Architecture

The system architecture of SecureShare follows a client-server model deployed on cloud infrastructure. The user interacts with the system through a web browser, which sends HTTP requests to the server.The application is hosted on an AWS EC2 instance running Linux. Nginx acts as the web server and handles incoming requests. When a user accesses the application, Nginx processes the request and forwards dynamic requests to the PHP engine.

PHP scripts handle the core logic of the application, including file uploads, password verification, expiry checks, and request processing. These scripts interact with the MariaDB database to store and retrieve file-related information.
Uploaded files are stored in a directory on the server, typically /usr/share/nginx/html/uploads. The database stores metadata such as file path, expiry time, and passcode, which are used to validate access.

The architecture ensures separation of concerns:

- Nginx handles web requests
- PHP handles logic
- Database handles storage
- File system stores actual files

10.Implementation

The implementation of SecureShare is divided into multiple modules, each responsible for a specific functionality.

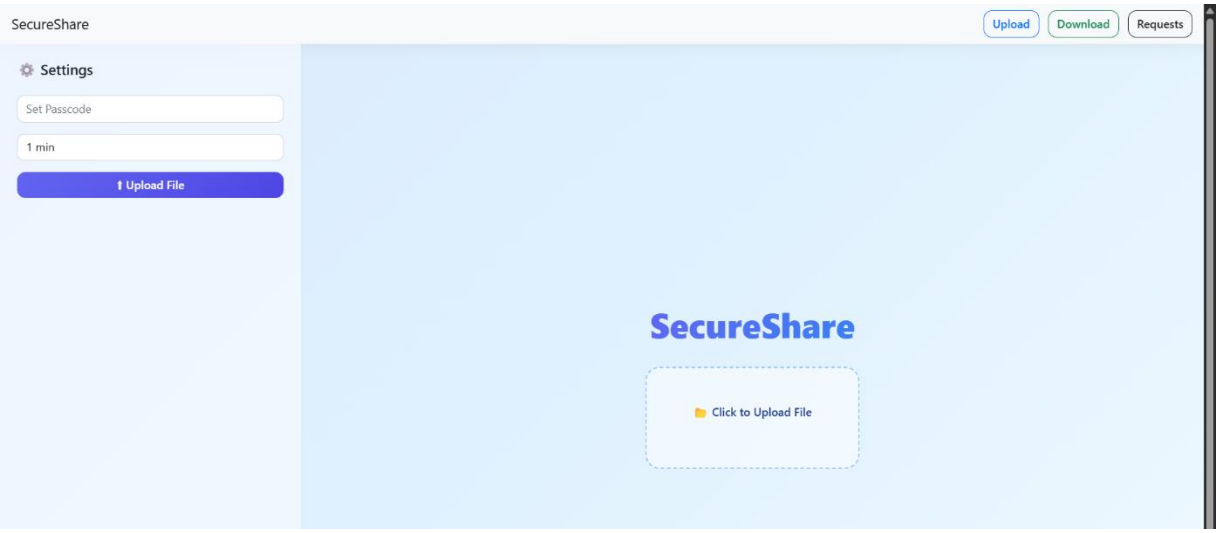
Module 1: File Upload:The user uploads a file through the web interface. The file is selected using a custom UI upload box. Along with the file, the user provides a passcode and selects an expiry time.The file is stored in the server's uploads directory with a unique filename to avoid conflicts. The file path, passcode, and expiry time are stored in the database.

Module 2: File Download
To download a file, the user enters the passcode. The system retrieves the latest file from the database and verifies the passcode.If the passcode matches and the file has not expired, the system sends the file to the user using HTTP headers, forcing a download.

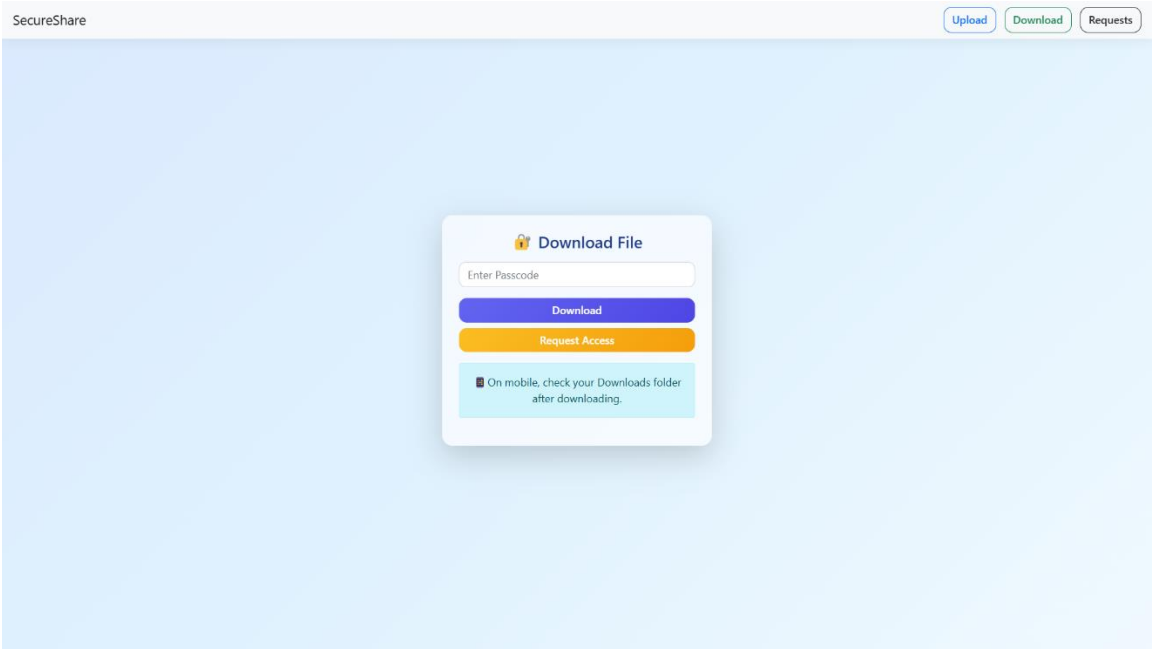
Module 3: Expiry Mechanism
Each file has an associated expiry time stored in the database. When a user attempts to download a file, the system checks whether the current time is less than the expiry time.
If the file has expired, access is denied, and the user is prompted to request access.

Module 4: Request Access
When a file expires, users can request access by clicking the "Request Access" button. The request is stored in the database with a status of "Pending".

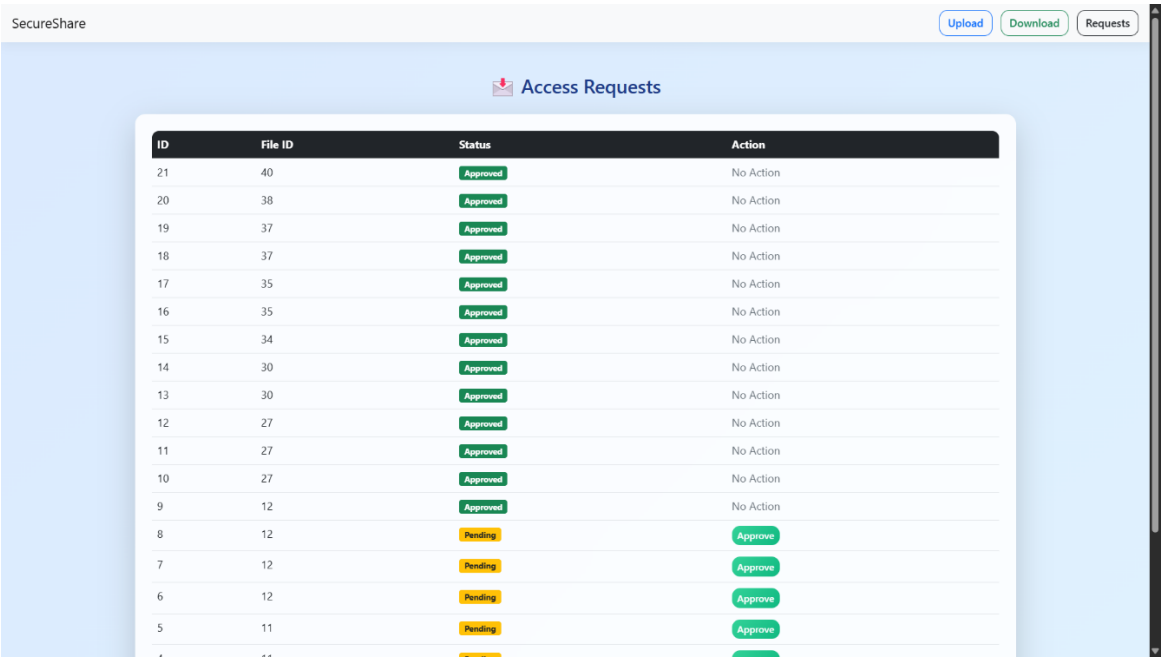
Module 5: Approval System
The admin views all requests on the requests page. Each request can be approved, which updates its status to "Approved".Upon approval, the expiry time of the file is extended (e.g., by 10 minutes), allowing the user to download the file again.



1.4 index.php



1.5 download.php



ID	File ID	Status	Action
21	40	Approved	No Action
20	38	Approved	No Action
19	37	Approved	No Action
18	37	Approved	No Action
17	35	Approved	No Action
16	35	Approved	No Action
15	34	Approved	No Action
14	30	Approved	No Action
13	30	Approved	No Action
12	27	Approved	No Action
11	27	Approved	No Action
10	27	Approved	No Action
9	12	Approved	No Action
8	12	Pending	Approve
7	12	Pending	Approve
6	12	Pending	Approve
5	11	Pending	Approve

1.6 requests.php

11.Results and Output

The results of the project demonstrate the successful implementation of a secure cloud-based file sharing system. The application was deployed on an AWS EC2 instance using a LEMP stack and was accessed through a public IP address.
The system was tested for various functionalities including file upload, password-protected download, expiry-based access control, and request-based access. All modules worked as expected, confirming the correctness and reliability of the system. The upload module allows users to select a file and define a passcode and expiry time. Upon successful upload, the file is stored in the server directory and its details are saved in the database.

The download module verifies the entered passcode and checks whether the file has expired. If both conditions are satisfied, the file is successfully downloaded. In case of incorrect passcode or expiry, appropriate messages are displayed. The request access module allows users to request access after expiry. These requests are displayed in the admin panel (requests page), where they can be approved. Upon approval, the file becomes accessible again for a limited duration.

12. Advantages

The SecureShare system offers several advantages over traditional file sharing methods. One of the primary advantages is enhanced security through password protection. Each file is accessible only to users who know the correct passcode, preventing unauthorized access. Another major advantage is the implementation of expiry-based access control. Files are not available indefinitely, reducing the risk of data misuse and ensuring that sensitive files are accessible only for a limited time.

The request-based access system adds flexibility and control. Even after expiry, users can request access, and the admin can approve or deny the request. This ensures that access is not permanently lost while still maintaining security.
The system is deployed on a cloud platform, making it accessible from anywhere with an internet connection. This increases availability and scalability.

The use of a LEMP stack ensures high performance and efficient handling of requests. Nginx provides fast response times, while PHP and MariaDB handle backend processing effectively. Additionally, the user interface is modern and responsive, providing a better user experience through animations, clear feedback, and intuitive design.

13.Future Scope

Although the current system is functional and secure, there are several areas for future improvement and enhancement. One possible enhancement is the integration of a domain name and HTTPS security. This would make the application more professional and secure for real-world usage. Another improvement is the addition of a user authentication system. Currently, the system does not distinguish between different users. Adding login and registration features would allow personalized file management. The system can also be extended to include email notifications. For example, users can be notified when their request is approved or when a file is about to expire.

A file preview feature can be implemented to allow users to view files (images, PDFs) before downloading them.Another enhancement is the implementation of drag-and-drop file upload and upload progress indicators, improving user experience. The system can also be scaled using load balancers and multiple EC2 instances to handle higher traffic.
Finally, a mobile application version can be developed to make the system more accessible on smartphones.

14.Conclusion

The SecureShare project successfully demonstrates the design and implementation of a secure file sharing system using cloud computing technologies. The system addresses key challenges in traditional file sharing by incorporating password protection, expiry-based access, and request-based control mechanisms.
By deploying the application on an AWS EC2 instance using a LEMP stack, the project provides practical exposure to cloud infrastructure and real-world deployment scenarios. It highlights the importance of cloud computing in building scalable and accessible applications. The modular design of the system ensures maintainability and scalability. Each component, including file upload, download, expiry control, and request handling, functions effectively and integrates seamlessly.

The project also emphasizes user experience by providing a clean and modern interface. Features such as dynamic file name display, responsive design, and clear feedback improve usability.

15. References

- AWS EC2 Documentation
- PHP Official Documentation
- Nginx Documentation
- MariaDB Documentation
- Bootstrap Documentation
- Online tutorials and guides related to LEMP stack.